

Image Recognition



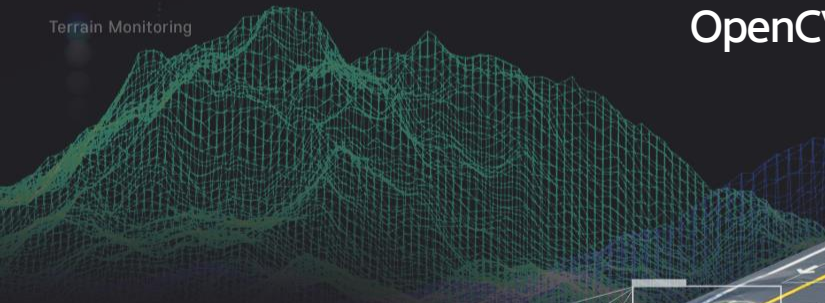
강원혁신플랫폼

컴퓨터 비전



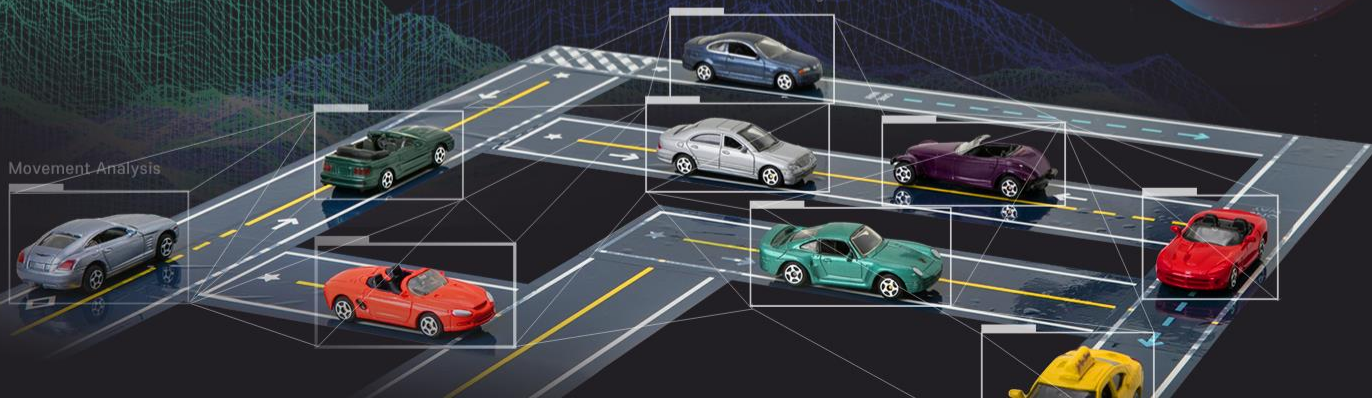
OpenCV로 영상 데이터 다루기

Terrain Monitoring



Autonomous Driving

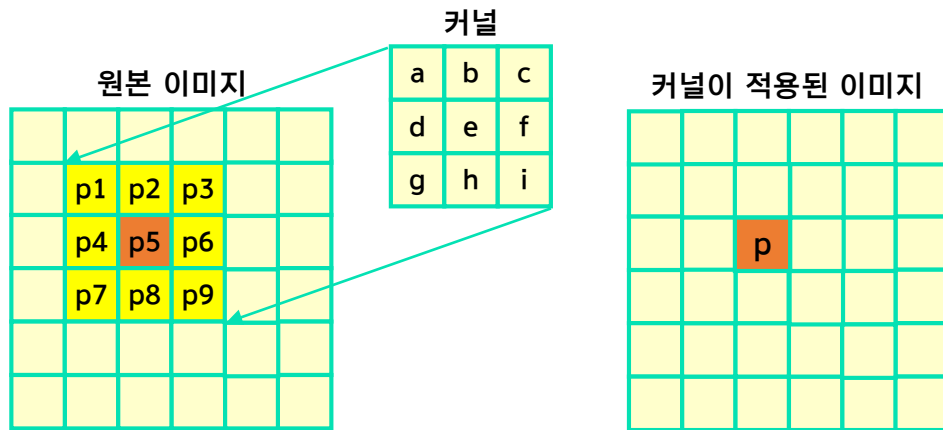
Movement Analysis



이미지에 필터를 씌워보기

이미지를 조작할 때 사용하는 방법 중에 **커널(Kernel)**이라고 불리는 작은 **행렬(Matrix)**을 사용하여 **필터링(Filtering)**하는 방법이 있음

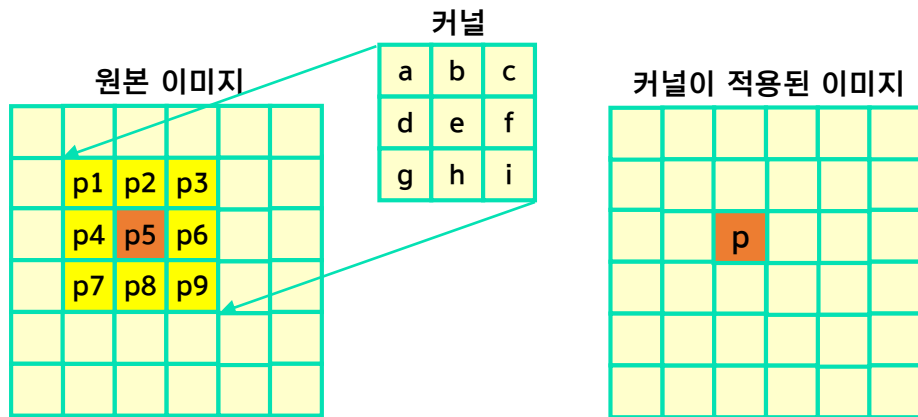
커널을 통해 이미지를 변형하는 것은 다음 그림과 같은 방식을 사용



$$p = p1*a + p2*b + p3*c + p4*d + p5*e + p6*f + p7*g + p8*h + p9*i$$



■ 커널 : 작은 크기의 행렬로, 원본 이미지에 덮여서 계산에 사용



$$p = p1*a + p2*b + p3*c + p4*d + p5*e + p6*f + p7*g + p8*h + p9*i$$

- ☞ 그림의 왼쪽 p5 픽셀의 값을 커널이 적용된 이미지의 p로 바꾸는 작업이 진행되고 있음
- ☞ 커널과 원본 이미지의 대응되는 값들이 서로 곱해지고, 모두 더하여 새로운 이미지의 픽셀값 p를 구하는 것임
- ☞ 커널을 움직여 가면서 이러한 작업을 모든 픽셀에 대해 적용하면 이미지 전체의 변경이 이루어짐

■ 간단한 필터링은 하나의 픽셀 값을 변경할 때 주위의 값을 평균을 취하는 방법

- 이렇게 할 경우 튀는 값이 사라지고 전체적으로 부드러운 이미지가 되는데, 이러한 처리를 이미지 흐림(Blur)이라고 함
- 3×3의 커널을 사용한다면 다음과 같은 커널을 사용할 수 있음

$$M = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

이 커널은 인접한 8 픽셀과 자기 자신의 평균(9개의 평균)을 내는 커널이 될 것임

```
kernal1 = np.ones((3,3), np.float32) / 9 # 주변 8개와 자기자신 픽셀과의 평균(9개의 평균)을 계산
kernal1
# array([[0.11111111, 0.11111111, 0.11111111],
#        [0.11111111, 0.11111111, 0.11111111],
#        [0.11111111, 0.11111111, 0.11111111]], dtype=float32)
```

다음은 3×3 행렬의 커널과 9×9 행렬의 커널을 생성하는 코드이다.

```
kernal1 = np.ones((3,3), np.float32) / 9 # 주변 8개와 자기자신 픽셀과의 평균(9개의 평균)을 계산
print(kernal1)
# [[0.11111111 0.11111111 0.11111111]
#  [0.11111111 0.11111111 0.11111111]
#  [0.11111111 0.11111111 0.11111111]]

kernal2 = np.ones((9,9), np.float32) / 81 # 주변 80개와 자기자신 픽셀과의 평균(81개의 평균)을 계산
print(kernal2)
# [[0.01234568 0.01234568 0.01234568 0.01234568 0.01234568 0.01234568 0.01234568 0.01234568 0.01234568]
# ...
```

- 위 코드에서 행렬을 만들 때 numpy 모듈의 ones() 함수를 이용함
- ones() 함수는 행렬의 모든 값을 1로 채움

커널을 특정 이미지에 적용하는 방법은 `filter2D()` 함수를 사용하면 된다.

다음은 `cv2` 모듈의 `filter2D()` 함수의 형식임

```
cv2.filter2D(src, ddepth, kernel[, dst[, anchor[, delta[, borderType]]]])
```

- ↻ `src` 인자는 변경할 이미지가 입력됨
- ↻ `ddepth` 인자는 이미지 깊이로 `-1`을 주면 주어진 입력 이미지와 동일한 값이 됨

다음은 3×3 행렬의 커널과 9×9 행렬의 커널을 적용하여 이미지를 변형하는 코드이다.

```
org = cv2.imread(os.getcwd()+'/images/mandrill.png', 1)
print(org.shape)           # (256, 256, 3)
print(org[:1, :1, :])     # [[[ 56 107 117]]]

averaged33 = cv2.filter2D(org, -1, kernal1)    # 3×3 행렬의 커널 적용
print(averaged33.shape)           # (256, 256, 3)
print(averaged33[:1, :1, :])     # [[[46 79 89]]]

averaged99 = cv2.filter2D(org, -1, kernal2)    # 9×9 행렬의 커널 적용
print(averaged99.shape)           # (256, 256, 3)
print(averaged99[:1, :1, :])     # [[[47 83 94]]]
```

☞ 위 코드 결과를 보면, 3×3 행렬의 커널과 9×9 행렬의 커널이 적용된 결과값이 다른 것을 볼 수 있음

다음은 맨드릴 원숭이 이미지에 3×3 행렬의 커널(9개 픽셀의 평균),
9×9 행렬의 커널(81개 픽셀의 평균)을 적용하는 코드이다.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

org = cv2.imread(os.getcwd()+'/images/mandrill.png', cv2.IMREAD_COLOR)
org_rgb = cv2.cvtColor(org, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
kernel1 = np.ones((3,3), np.float32) / 9 # 3×3 행렬의 커널(9개 픽셀의 평균)
kernel2 = np.ones((9,9), np.float32) / 81 # 9×9 행렬의 커널(81개 픽셀의 평균)
averaged33 = cv2.filter2D(org, -1, kernel1) # 3×3 행렬의 커널 적용
averaged99 = cv2.filter2D(org, -1, kernel2) # 9×9 행렬의 커널 적용
averaged33_rgb = cv2.cvtColor(averaged33, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
averaged99_rgb = cv2.cvtColor(averaged99, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
```

```
plt.figure(figsize=(10, 10))
plt.subplot(131)
plt.imshow(org_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(132)
plt.imshow(averaged33_rgb)
plt.title("averaged33")
plt.xticks([])
plt.yticks([])
plt.subplot(133)
plt.imshow(averaged99_rgb)
plt.title("averaged99")
plt.xticks([])
plt.yticks([])
plt.show()
```

커널 적용하는 부분

시각화하는 부분

다음은 맨드릴 원숭이 이미지에 3×3 행렬의 커널(9개 픽셀의 평균), 9×9 행렬의 커널(81개 픽셀의 평균)을 적용하는 코드이다.

3x3 행렬의 커널과 9x9 행렬의 커널을 적용하는 부분

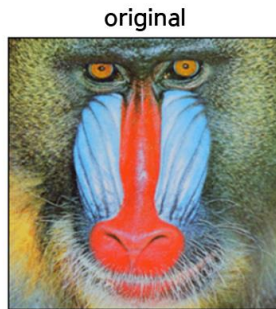
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

org = cv2.imread(os.getcwd()+'/images/mandrill.png', cv2.IMREAD_COLOR)
org_rgb = cv2.cvtColor(org, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
kernel1 = np.ones((3,3), np.float32) / 9 # 3×3 행렬의 커널(9개 픽셀의 평균)
kernel2 = np.ones((9,9), np.float32) / 81 # 9×9 행렬의 커널(81개 픽셀의 평균)
averaged33 = cv2.filter2D(org, -1, kernel1) # 3×3 행렬의 커널 적용
averaged99 = cv2.filter2D(org, -1, kernel2) # 9×9 행렬의 커널 적용
averaged33_rgb = cv2.cvtColor(averaged33, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
averaged99_rgb = cv2.cvtColor(averaged99, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
```

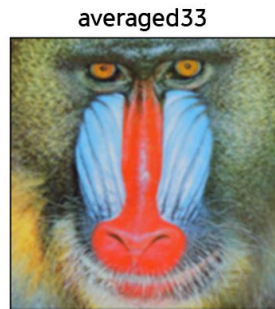
다음은 맨드릴 원숭이 이미지에 3×3 행렬의 커널(9개 픽셀의 평균),
 9×9 행렬의 커널(81개 픽셀의 평균)을 적용하는 코드이다.

```
plt.figure(figsize=(10, 10))
plt.subplot(131)
plt.imshow(org_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(132)
plt.imshow(averaged33_rgb)
plt.title("averaged33")
plt.xticks([])
plt.yticks([])
plt.subplot(133)
plt.imshow(averaged99_rgb)
plt.title("averaged99")
plt.xticks([])
plt.yticks([])
plt.show()
```

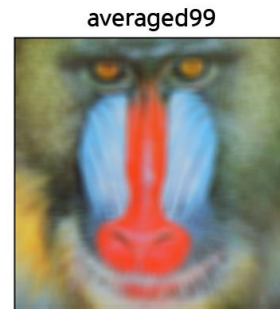
시각화하는 부분



원본 이미지



이미지 변형
(3×3 행렬의 커널)



이미지 변형
(9×9 행렬의 커널)

아래 그림에서 두 커널을 적용한 결과를 비교해 보자.

original



원본 이미지

averaged33



이미지 변형
(3×3 행렬의 커널)

averaged99



이미지 변형
(9×9 행렬의 커널)



■ 상자 필터(Box filter)

🔄 필터 중에서 특정 픽셀 주변의 픽셀들이 가진 값의 평균을 취하는 커널

■ `blur(image, filter_size)` 함수를 통해 자주 사용되는 필터들을 적용할 수 있음

🔄 `filter_size` 인자는 튜플로 주어짐

예 `cv2.blur(image, (n,n))`를 적용할 경우와 `np.ones((n, n), np.float32) / (n*n)` 함수를 이용하여 커널을 적용하는 것은 동일한 결과를 만들어 냄

```
kernel = cv2.blur(image, (n, n))
```

```
kernel = np.ones((n, n), np.float32) / (n*n)
```

위의 두 함수는 동일한 결과를 만들어 냄

다음은 맨드릴 원숭이 이미지에 cv2 모듈의 `blur()` 함수와 numpy 모듈의 `ones()` 함수를 이용한 필터를 적용하는 코드이다.

```
org = cv2.imread(os.getcwd()+'/images/mandrill.png', cv2.IMREAD_COLOR)
org_rgb = cv2.cvtColor(org, cv2.COLOR_BGR2RGB) # RGB 모드로 변환

n = 3
kernal_blur = cv2.blur(org, (n, n))
kernal_one = np.ones((n, n), np.float32) / (n*n)
kernal_one_filter = cv2.filter2D(org, -1, kernal_one)
kernal_blur_rgb = cv2.cvtColor(kernal_blur, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
kernal_one_filter_rgb = cv2.cvtColor(kernal_one_filter, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
```

필터 적용하는 부분

```
plt.figure(figsize=(10, 10))
plt.subplot(131)
plt.imshow(org_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(132)
plt.imshow(kernal_blur_rgb)
plt.title("kernal_blur")
plt.xticks([])
plt.yticks([])
plt.subplot(133)
plt.imshow(kernal_one_filter_rgb)
plt.title("kernal_one_filter")
plt.xticks([])
plt.yticks([])
plt.show()
```

시각화하는 부분

다음은 맨드릴 원숭이 이미지에 cv2 모듈의 `blur()` 함수와 numpy 모듈의 `ones()` 함수를 이용한 필터를 적용하는 코드이다.

필터 적용하는 부분

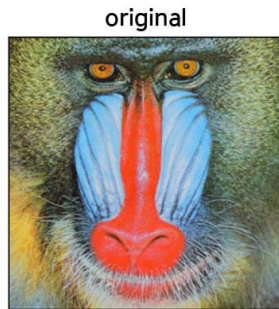
```
org = cv2.imread(os.getcwd()+'/images/mandrill.png', cv2.IMREAD_COLOR)
org_rgb = cv2.cvtColor(org, cv2.COLOR_BGR2RGB) # RGB 모드로 변환

n = 3
kernal_blur = cv2.blur(org, (n, n))
kernal_one = np.ones((n, n), np.float32) / (n*n)
kernal_one_filter = cv2.filter2D(org, -1, kernal_one)
kernal_blur_rgb = cv2.cvtColor(kernal_blur, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
kernal_one_filter_rgb = cv2.cvtColor(kernal_one_filter, cv2.COLOR_BGR2RGB) # RGB 변환
```

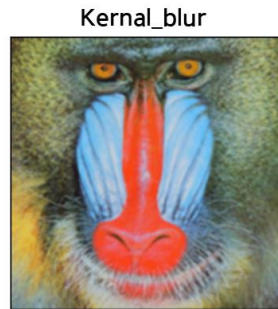
다음은 맨드릴 원숭이 이미지에 cv2 모듈의 `blur()` 함수와 numpy 모듈의 `ones()` 함수를 이용한 필터를 적용하는 코드이다.

```
plt.figure(figsize=(10, 10))
plt.subplot(131)
plt.imshow(org_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(132)
plt.imshow(kernal_blur_rgb)
plt.title("kernal_blur")
plt.xticks([])
plt.yticks([])
plt.subplot(133)
plt.imshow(kernal_one_filter_rgb)
plt.title("kernal_one_filter")
plt.xticks([])
plt.yticks([])
plt.show()
```

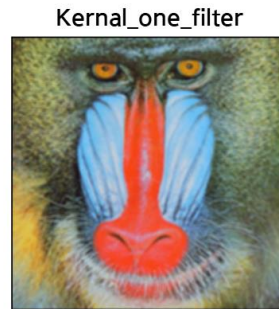
시각화하는 부분



원본 이미지

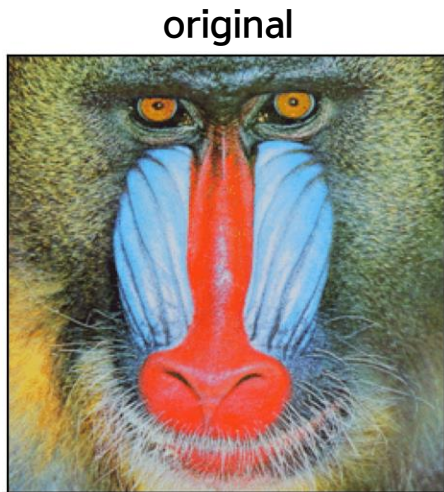


이미지 변형
(blur() 함수를 이용한
3×3 행렬의 커널)

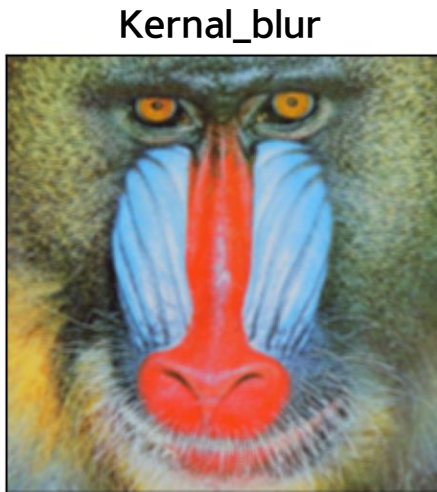


이미지 변형
(ones() 함수를 이용한
3×3 행렬의 커널)

아래 그림에서 동일한 것을 볼 수 있으며,
즉, cv2 모듈의 `blur()` 함수와 numpy 모듈의 `ones()` 함수를 이용한 필터는 동일하다.



원본 이미지



이미지 변형
(`blur()` 함수를 이용한
3×3 행렬의 커널)

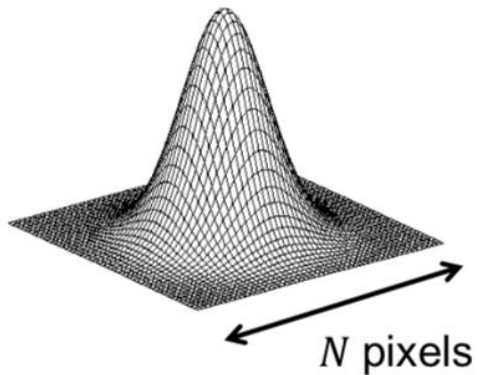


이미지 변형
(`ones()` 함수를 이용한
3×3 행렬의 커널)

OpenCV로 이미지 필터링하기

OpenCV에는 blur() 함수 뿐만이 아니라 다양한 필터를 적용할 수 있는 함수들이 있음

- 특히, 이미지의 잡음(Noise)을 줄이는 블러링(Blurring)을 위해서 여러 종류의 필터를 적용할 수 있음



가우시안 흐름(Gaussian blur)

가우스(Gauss) 필터를 적용하는 대표적인 필터

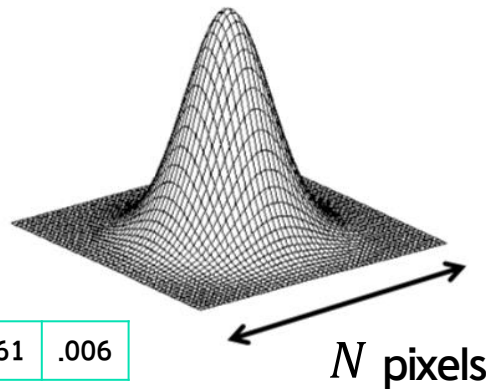
커널의 중심에서 얼마나 떨어져 있는지 만을 고려하여
가중치가 결정되는 공간 커널(Space kernel)이라고 할 수 있음

■ 가우스 필터

- 커널 영역의 모든 커널에 동일한 중요도를 부여하는 것이 아니라 **중심 픽셀**에는 **더 높은 중요도**를 부여하고, **중심에서 멀어질수록 중요도를 낮게** 하여 **더 합**
- 이때 이 **중요도**를 **가중치(weight)**라고 부르고, 이렇게 **더하는 것을 가중합(weighted sum)**이라고 함
- 이 **가중치**를 **가우스 함수**를 통해 **결정하는 것이 가우스 필터**임

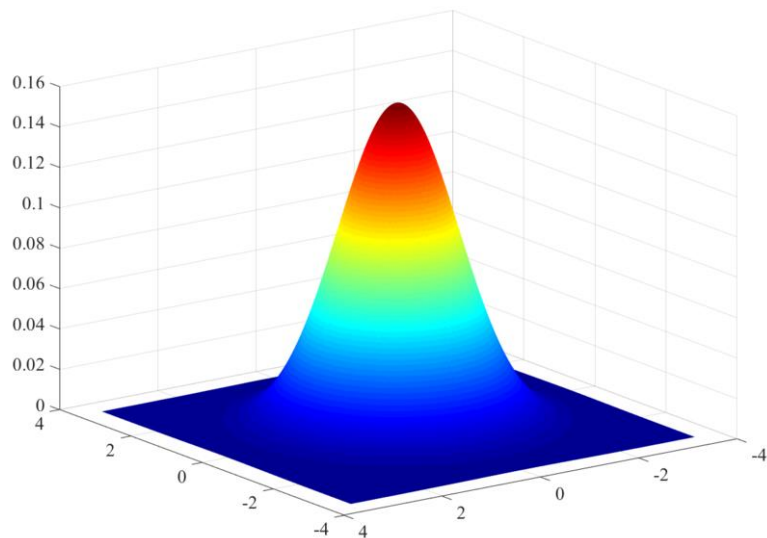
1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

.006	.061	.242	.383	.242	.061	.006
------	------	------	------	------	------	------



■ 가우스 함수

🔄 종 모양(Bell shaped) 함수 혹은 정규분포(Normal distribution) 함수로 잘 알려져 있음



실제 이미지에 적용할 때는 연속함수를
적용하는 것이 아니라 픽셀 공간에서
정해지는 이산적(Discrete) 근사치를 사용



- 예를 들어 5×5 크기의 가우시안 커널은 다음과 같은 값을 가짐

$$K_{Gaussian} = \frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$

- ② 이런 커널을 3×3이나 5×5 등의 크기로 일일이 만들어 쓰는 것은 매우 성가신 일이 될 것임
- ② OpenCV는 이러한 가우스 필터를 자동으로 적용할 수 있는 GaussianBlur() 함수를 제공함

다음은 cv2 모듈의 GaussianBlur() 함수이다.

```
dst = cv2.GaussianBlur(src, ksize, sigmaX[, dst[, sigmaY[, borderType]]])
```

- src 인자는 입력 이미지임
- ksize 인자는 커널의 크기이고, 이 값에 n을 인자로 주면 $n \times n$ 크기의 커널을 가우스 함수에 근거하여 자동으로 생성함
- sigmaX 인자는 x축 방향으로의 정규분포 표준편차(standard deviation)이고, 이 값에 따라 가우스 함수의 모양이 달라짐
- dst 인자는 결과가 출력될 이미지임
- sigmaY 인자는 y축 표준편차인데 기본값은 0이고, 이 값이 0이면 x축 표준편차와 같은 값으로 처리됨



다음은 맨드릴 원숭이 이미지에 cv2 모듈의 `GaussianBlur()` 함수를 이용한 가우스 필터를 적용하는 코드이다.

```
org = cv2.imread(os.getcwd()+'/images/mandrill.png', cv2.IMREAD_COLOR)
org_rgb = cv2.cvtColor(org, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
gaussian33 = cv2.GaussianBlur(org, (3,3), 1) # n=3, x축 표준편차=1
gaussian99 = cv2.GaussianBlur(org, (9,9), 1) # n=9, x축 표준편차=1
gaussian33_rgb = cv2.cvtColor(gaussian33, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
gaussian99_rgb = cv2.cvtColor(gaussian99, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
```

필터 적용하는 부분

```
plt.figure(figsize=(10, 10))
plt.subplot(131)
plt.imshow(org_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(132)
plt.imshow(gaussian33_rgb)
plt.title("gaussian33")
plt.xticks([])
plt.yticks([])
plt.subplot(133)
plt.imshow(gaussian99_rgb)
plt.title("gaussian99")
plt.xticks([])
plt.yticks([])
plt.show()
```

시각화하는 부분

다음은 맨드릴 원숭이 이미지에 cv2 모듈의 GaussianBlur() 함수를 이용한
가우스 필터를 적용하는 코드이다.

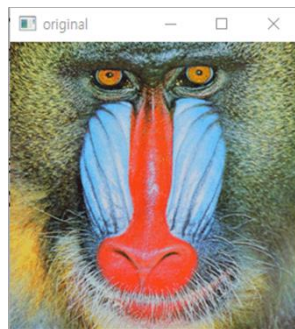
필터 적용하는 부분

```
org = cv2.imread(os.getcwd()+'/images/mandrill.png', cv2.IMREAD_COLOR)
org_rgb = cv2.cvtColor(org, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
gaussian33 = cv2.GaussianBlur(org, (3,3), 1) # n=3, x축 표준편차=1
gaussian99 = cv2.GaussianBlur(org, (9,9), 1) # n=9, x축 표준편차=1
gaussian33_rgb = cv2.cvtColor(gaussian33, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
gaussian99_rgb = cv2.cvtColor(gaussian99, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
```

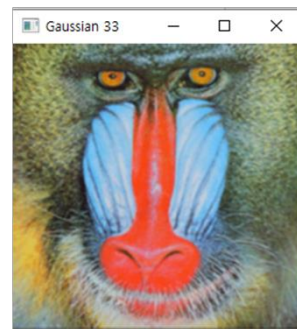
다음은 맨드릴 원숭이 이미지에 cv2 모듈의 GaussianBlur() 함수를 이용한
가우스 필터를 적용하는 코드이다.

```
plt.figure(figsize=(10, 10))
plt.subplot(131)
plt.imshow(org_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(132)
plt.imshow(gaussian33_rgb)
plt.title("gaussian33")
plt.xticks([])
plt.yticks([])
plt.subplot(133)
plt.imshow(gaussian99_rgb)
plt.title("gaussian99")
plt.xticks([])
plt.yticks([])
plt.show()
```

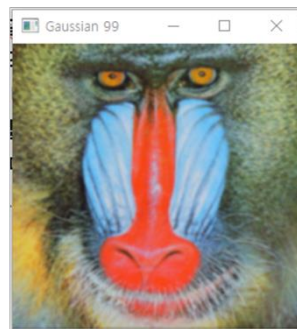
시각화하는 부분



원본 이미지



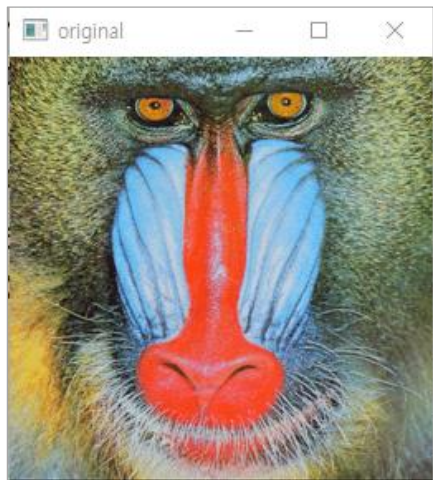
이미지 변형
(3×3 행렬의 가우스 커널)



이미지 변형
(9×9 행렬의 가우스 커널)

실행 결과를 살펴보면 이전에 비해서 부드러운 느낌이 들고,
원본 이미지의 상이 크게 왜곡 없이 나타남을 확인할 수 있다.

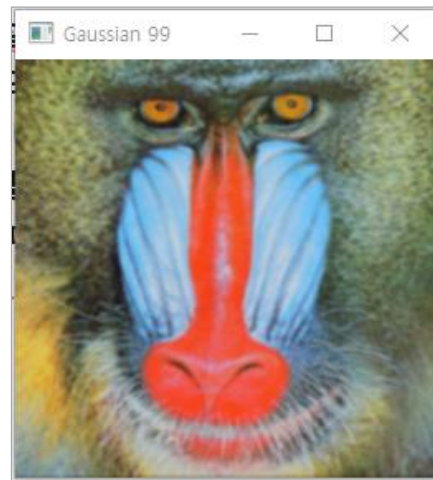
즉, 특정 픽셀 주변의 평균을 취하는 상자 필터(Box filter)보다 부드러운 느낌이 듭니다.



원본 이미지



이미지 변형
(3×3 행렬의 가우스 커널)



이미지 변형
(9×9 행렬의 가우스 커널)

다양한 흐림 필터로 잡음을 제거하기

이미지를 원본보다 더 흐리게 만드는 일은 잡음을 제거하는 방법으로 널리 사용

OpenCV는 상자 필터와 가우스 필터 뿐만 아니라
중앙값 필터(medianBlur()), 양방향 필터(bilateralFilter()) 같은 다양한 흐림(blur) 기능을 제공

01

가우시안 필터
(Gaussian filter)

02

중앙값 필터
(Median filter)

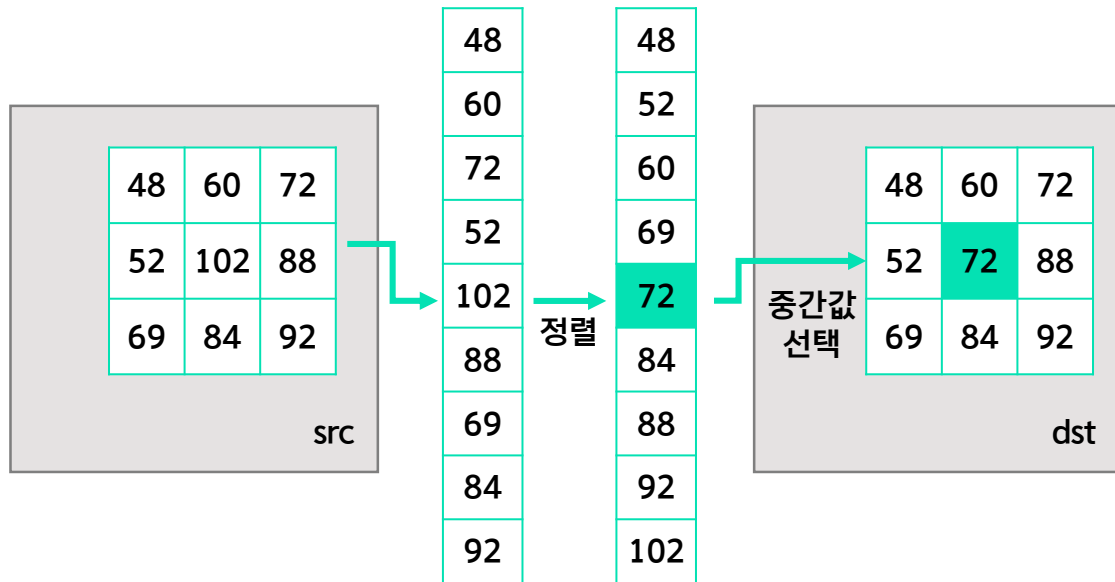
03

양방향 필터
(Bilateral filter)

중앙값 필터(Median filter)



- 주변 픽셀들의 값들을 정렬하여 그 중앙값(Median)으로 픽셀 값을 대체함





중앙값 필터(Median filter)

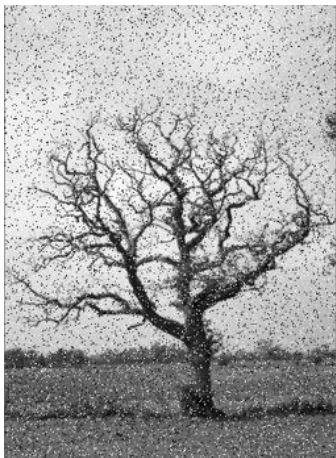


- OpenCV의 **medianBlur()** 함수는 **중앙값 흐림(median blur)**을 수행하는 함수로 **두 개의 매개변수를 가짐**
 - 🕒 사용 방법 : `cv2.medianBlur(src, ksize[, dst])` → dst
 - 🕒 src : **입력 영상**, 각 채널 별로 처리
 - 🕒 ksize : **커널 크기**, **1보다 큰 홀수를 지정**, $n \times n$ 크기의 필터 영역에서 **중앙값(Median)**을 찾아 **그 값으로 픽셀을 설정하는 것**
 - 🕒 dst : **출력 영상**, src와 같은 크기, 같은 타입

중앙값 필터(Median filter)



- 이 흐림(Blur) 효과는 이미지에서 주위에 비해 튀는 작은 점들을 지우는 데에 효과적임
 - ➡ 이런 작은 점과 같은 효과는 이미지에서 주위에 비해 튀는 작은 점들을 지우는 데에 효과적임
 - ➡ 소금과 후추(Salt and pepper) 잡음 : 아래 그림과 같이 작은 점과 같은 잡음



다음은 cv2 모듈의 `medianBlur()` 함수를 이용한 흐림 처리를 한 뒤,
결과를 비교해 보는 코드이다.

```
import cv2, os
import matplotlib.pyplot as plt
```

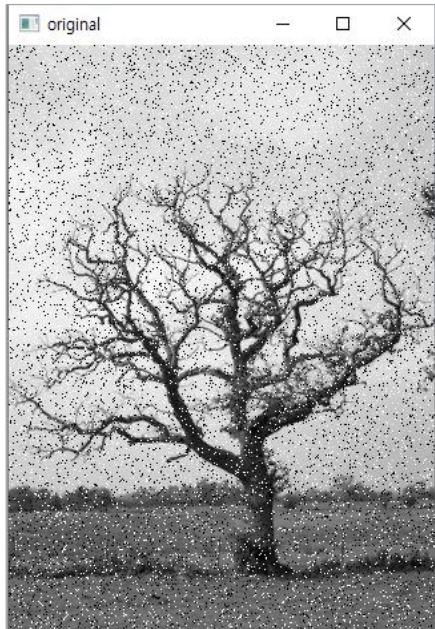
필터 적용하는 부분

```
org_img = cv2.imread(os.getcwd()+'/images/salt_pepper.png', 0)
org_rgb = cv2.cvtColor(org_img, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
salt_medBlur_img = cv2.medianBlur(org_rgb, 5) # 중앙값 흐림(5x5 크기의 필터)
salt_medBlur_rgb = cv2.cvtColor(salt_medBlur_img, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
```

```
plt.figure(figsize=(10, 10))
plt.subplot(121)
plt.imshow(org_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(122)
plt.imshow(salt_medBlur_img)
plt.title("salt_medianBlur")
plt.xticks([])
plt.yticks([])
plt.show()
```

시각화하는 부분

아래 그림과 같이 원본 이미지에서 잡음이 제거된 것을 볼 수 있다.



원본 이미지



중앙값 흐림 필터 적용 이미지

양방향 필터(Bilateral filter)



- 가우스 필터는 커널의 중심에서 얼마나 떨어져 있는지 만을 고려하여 가중치가 결정되는 공간 커널(space kernel)이라고 할 수 있음
- OpenCV의 `bilateralFilter()` 함수는 두 개의 기준을 고려하는 양방향 필터임
- `bilateralFilter()` 함수는 이러한 공간 커널에 픽셀의 색상 값이 얼마나 차이가 나는지를 고려하는 필터를 하나 더 쓰는 것임
 - ↻ 양방향 커널은 위치와 무관하게 중심 픽셀의 색과 해당 픽셀의 색 차이에 의해 값이 결정됨
 - ↻ 따라서, 양방향 필터를 사용하면 잡음은 제거하면서 이미지의 특징은 더 잘 유지할 수 있게 됨

다음은 양방향 필터를 위한 cv2 모듈의 `bilateralFilter()` 함수의 사용 방법이다.

```
dst = cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace[, dst[, borderType]])
```

- ➡ `src` 인자는 원본 이미지임(8비트 또는 실수형, 1채널 또는 3채널)
- ➡ `d` 인자는 필터링에 사용될 이웃 픽셀의 거리(지름), 음수(-1)를 입력하면 `sigmaSpace` 값에 의해 자동 결정(권장)됨
- ➡ `sigmaColor` 인자는 색 공간에서 필터의 표준편차임(엣지나 아니냐를 판단하는 기준)
- ➡ `sigmaSpace` 인자는 좌표 공간에서 필터의 표준편차임(100으로 주어주면 가우시안 필터를 적용하는 것과 동일)
- ➡ `dst` 인자는 출력 영상임(src와 같은 크기, 같은 타입)
- ➡ `borderType` 인자는 가장자리 픽셀 처리방식임

다음은 cv2 모듈의 `bilateralFilter()` 함수로(양방향 필터 적용)
흐림 처리 후 결과를 비교해 보는 코드이다.

```
import cv2, sys, os
import matplotlib.pyplot as plt

org = cv2.imread(os.getcwd()+'/images/lena_color.tiff')
org_rgb = cv2.cvtColor(org, cv2.COLOR_BGR2RGB) # RGB 모드로 변환

if org is None:
    print('Image load failed!')
    sys.exit()

dst_img = cv2.bilateralFilter(org, -1, 10, 5) # 양방향 필터
dst_rgb = cv2.cvtColor(dst_img, cv2.COLOR_BGR2RGB) # RGB 모드로 변환

plt.figure(figsize=(10, 10))
plt.subplot(121)
plt.imshow(org_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(122)
plt.imshow(dst_rgb)
plt.title("bilateralFilter")
plt.xticks([])
plt.yticks([])
plt.show()
```

필터 적용하는 부분

시각화하는 부분

다음은 cv2 모듈의 `bilateralFilter()` 함수로(양방향 필터 적용)
흐림 처리 후 결과를 비교해 보는 코드이다.

필터 적용하는 부분

```
import cv2, sys, os
import matplotlib.pyplot as plt

org = cv2.imread(os.getcwd()+'/images/lena_color.tiff')
org_rgb = cv2.cvtColor(org, cv2.COLOR_BGR2RGB)    # RGB 모드로 변환

if org is None:
    print('Image load failed!')
    sys.exit()

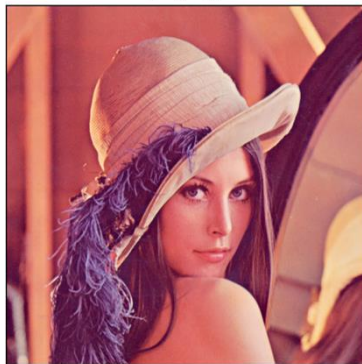
dst_img = cv2.bilateralFilter(org, -1, 10, 5)    # 양방향 필터
dst_rgb = cv2.cvtColor(dst_img, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
```

다음은 cv2 모듈의 `bilateralFilter()` 함수로(양방향 필터 적용)
흐림 처리 후 결과를 비교해 보는 코드이다.

시각화하는 부분

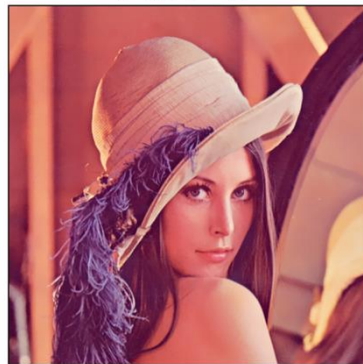
```
plt.figure(figsize=(10, 10))  
plt.subplot(121)  
plt.imshow(org_rgb)  
plt.title("original")  
plt.xticks([])  
plt.yticks([])  
plt.subplot(122)  
plt.imshow(dst_rgb)  
plt.title("bilateralFilter")  
plt.xticks([])  
plt.yticks([])  
plt.show()
```

original



원본 이미지

bilateralFilter



양방향 필터 적용 이미지

아래 그림과 같이 원본 이미지에 양방향 필터가 적용된 것을 볼 수 있다.

original



원본 이미지

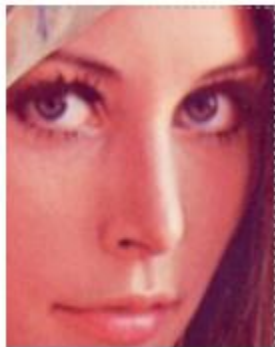
bilateralFilter



양방향 필터 적용 이미지

■ 다양한 흐림(Blur) 효과

- 가우시안(Gaussian), 중앙값(Median) 등의 다양한 흐림 효과가 계산의 빠르기, 튼튼한 값 제거 등과 같은 각각의 장점을 가지고 있음
- 하지만, 이미지의 원래 특성을 그대로 유지한다는 점에서 양방향 필터(BilateralFilter)는 이미지 인식 등의 분야에서 폭넓은 활용도를 갖고 있는 필터임



original



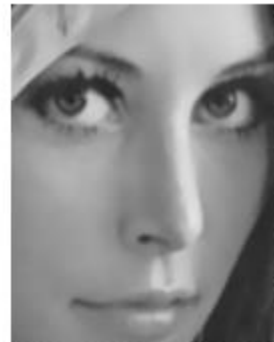
Box filter



Gaussian Filter



Bilateral Filter(d=-1)



Bilateral Filter(d=3)

다음은 가우시안(Gaussian), 중앙값(Median), 양방향 필터(BilateralFilter) 흐림(Blur)을 이용한 효과를 비교해 보자.

구현을 위해 cv2 모듈의 medianBlur(), GaussianBlur(), bilateralFilter() 함수를 이용함

```
org_img = cv2.imread(os.getcwd()+'/images/mandrill.png', 1)
org_rgb = cv2.cvtColor(org_img, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
gaussian_image = cv2.GaussianBlur(org_img, (9,9), 1) # 가우시안 흐림: n=9, x축 표준편차=1
median_image = cv2.medianBlur(org_img, 9) # 중앙값 흐림: 9x9
bilateral_image = cv2.bilateralFilter(org_img, -1, 10, 5) # 양방향 필터:
gaussian_rgb = cv2.cvtColor(gaussian_image, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
median_rgb = cv2.cvtColor(median_image, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
bilateral_rgb = cv2.cvtColor(bilateral_image, cv2.COLOR_BGR2RGB) # RGB 모드로 변환

plt.figure(figsize=(10, 10))
plt.subplot(141)
plt.imshow(org_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(142)
plt.imshow(gaussian_rgb)
plt.title("gaussian")
plt.xticks([])
plt.yticks([])
plt.subplot(143)
plt.imshow(median_rgb)
plt.title("median")
plt.xticks([])
plt.yticks([])
plt.subplot(144)
plt.imshow(bilateral_rgb)
plt.title("bilateral")
plt.xticks([])
plt.yticks([])
plt.show()
```

필터 적용하는 부분

시각화하는 부분

다음은 가우시안(Gaussian), 중앙값(Median), 양방향 필터(BilateralFilter) 흐림(Blur)을 이용한 효과를 비교해 보자.

구현을 위해 cv2 모듈의 medianBlur(), GaussianBlur(), bilateralFilter() 함수를 이용함

```
org_img = cv2.imread(os.getcwd()+'/images/mandrill.png', 1)
org_rgb = cv2.cvtColor(org_img, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
gaussian_image = cv2.GaussianBlur(org_img, (9,9), 1) # 가우시안 흐림 : n=9, x축 표준편차=1
median_image = cv2.medianBlur(org_img, 9) # 중앙값 흐림 : 9x9
bilateral_image = cv2.bilateralFilter(org_img, -1, 10, 5) # 양방향 필터 :
gaussian_rgb = cv2.cvtColor(gaussian_image, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
median_rgb = cv2.cvtColor(median_image, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
bilateral_rgb = cv2.cvtColor(bilateral_image, cv2.COLOR_BGR2RGB) # RGB 모드로 변환
plt.figure(figsize=(10, 10))
plt.subplot(141)
plt.imshow(org_rgb)
plt.title("original")
plt.xticks([])
plt.yticks([])
plt.subplot(142)
```

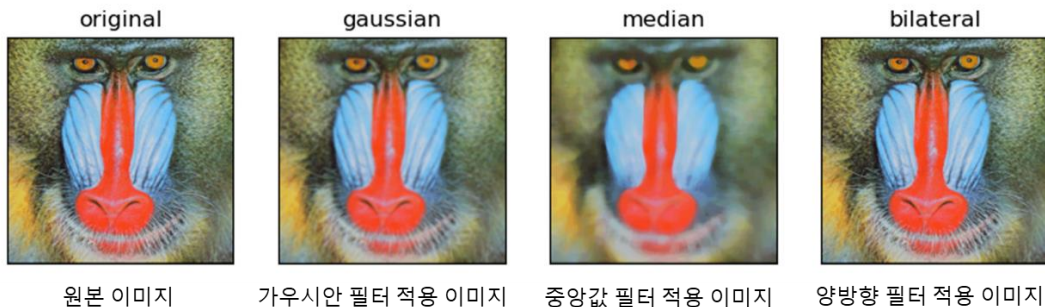
필터 적용하는 부분

다음은 가우시안(Gaussian), 중앙값(Median), 양방향 필터(BilateralFilter) 흐림(Blur)을 이용한 효과를 비교해 보자.

구현을 위해 cv2 모듈의 medianBlur(), GaussianBlur(), bilateralFilter() 함수를 이용함

```
plt.imshow(gaussian_rgb)
plt.title("gaussian")
plt.xticks([])
plt.yticks([])
plt.subplot(143)
plt.imshow(median_rgb)
plt.title("median")
plt.xticks([])
plt.yticks([])
plt.subplot(144)
plt.imshow(bilateral_rgb)
plt.title("bilateral")
plt.xticks([])
plt.yticks([])
plt.show()
```

시각화하는 부분



아래 그림에서 양방향 필터(BilateralFilter)가 적용한 이미지가
원래 이미지의 특성을 그대로 유지하고 있는 것을 볼 수 있다.

original



원본 이미지

gaussian



가우시안 필터 적용 이미지

median



중앙값 필터 적용 이미지

bilateral



양방향 필터 적용 이미지

이미지에서 관심 있는 곳만 남겨보기

이미지를 다룰 때, 특정한 임계값(Threshold) 이상인 값은 유효한 값, 그렇지 않은 값은 유효하지 않은 값으로 작업할 필요가 있는데, 이러한 작업을 이진화(Binarization)라고 함

- ➡ 이를 위해서 OpenCV의 `threshold()` 함수를 사용할 수 있는데 이 함수는 이진화를 할 수도 있고, 특정 임계치를 넘는 값은 일정한 값으로 모두 바꾸는 일을 할 수도 있음
- ➡ 이진화 처리는 간단하지만, 쉽지 않은 문제를 가지고 있음

이미지에서 관심 있는 곳만 남겨보기

이미지를 다룰 때, 특정한 임계값(Threshold) 이상인 값은 유효한 값, 그렇지 않은 값은 유효하지 않은 값으로 작업할 필요가 있는데, 이러한 작업을 **이진화(Binarization)**라고 함

이진화(Binarization)

영상을 **흑/백**으로 분류하여 처리하는 것

- ⦿ 이때 기준이 되는 **임계값**을 어떻게 결정할 것인지가 중요한 문제가 됨
- ⦿ **임계값**보다 **크면 백**, 작으면 **흑**이 됨
- ⦿ 기본 **임계처리**는 사용자가 **고정된 임계값**을 결정하고, 그 결과를 보여주는 단순한 형태임

다음은 cv2.threshold() 함수의 설명이다.

```
cv2.threshold(src, thresh, maxval, type) → retval, dst
```

- src : Input image로 Single-channel 이미지(Grayscale 이미지)
- thresh : 임계값
- maxval : 임계값을 넘었을 때 적용할 값(Value)
(이진화를 하여 픽셀을 최대값으로 바꿀 때 이 값이 사용됨)
- type : Thresholding type(임계치 적용 방식을 결정)

다음은 cv2.threshold() 함수의 설명이다.

```
cv2.threshold(src, thresh, maxval, type) → retval, dst
```

임계치 적용방식은 다음과 같이 선택이 가능함

Thresh type	의미
THRESH_BINARY	조건을 만족하는 픽셀 값을 최댓값으로 만족하지 않는 픽셀 값을 0(검정색)으로 설정
THRESH_BINARY_INV	조건을 만족하는 픽셀 값을 0으로, 만족하지 않는 픽셀 값을 최댓값으로 설정
THRESH_TRUNC	조건을 만족하는 픽셀 값을 최댓값으로, 만족하지 않으면 원본 그대로 둠
THRESH_TOZERO	조건을 만족하는 픽셀 값을 원본 값으로, 만족하지 않는 픽셀 값을 0으로 설정
THRESH_TOZERO_INV	조건을 만족하는 픽셀 값을 0으로, 만족하지 않는 픽셀 값을 원본 값으로 설정

다음은 cv2 모듈의 `threshold()` 함수를 사용해 임계치로 회색조 이미지를 이진화하여 비교해 보자.

여기서는 임계값 = 127, 임계값을 넘었을 때 적용할 값은 255로 함

```
import cv2, sys, os
import numpy as np
from matplotlib import pyplot as plt

org_img = cv2.imread(os.getcwd()+'/images/gray_gradient.jpg')

if org_img is None:
    print('Image load failed!')
    sys.exit()

ret, thresh1 = cv2.threshold(org_img, 127, 255, cv2.THRESH_BINARY)
ret, thresh2 = cv2.threshold(org_img, 127, 255, cv2.THRESH_BINARY_INV)
ret, thresh3 = cv2.threshold(org_img, 127, 255, cv2.THRESH_TRUNC)
ret, thresh4 = cv2.threshold(org_img, 127, 255, cv2.THRESH_TOZERO)
ret, thresh5 = cv2.threshold(org_img, 127, 255, cv2.THRESH_TOZERO_INV)

titles = ['Original', 'BINARY', 'BINARY_INV', 'TRUNC', 'TOZERO', 'TOZERO_INV']
images = [org_img, thresh1, thresh2, thresh3, thresh4, thresh5]

plt.figure(figsize=(15,15))
for i in range(6):
    plt.subplot(1,6,i+1), plt.imshow(images[i], 'gray'), plt.title(titles[i]), plt.xticks([]), plt.yticks([])

plt.show()
```

임계치 적용방식을 다르게 적용

다음은 cv2 모듈의 threshold() 함수를 사용해 임계치로 회색조 이미지를 이진화하여 비교해 보자.

여기서는 임계값 = 127, 임계값을 넘었을 때 적용할 값은 255로 함

```
import cv2, sys, os
import numpy as np
from matplotlib import pyplot as plt
```

```
org_img = cv2.imread(os.getcwd()+'/images/gray_gradient.jpg')
```

```
if org_img is None:
    print('Image load failed!')
    sys.exit()
```

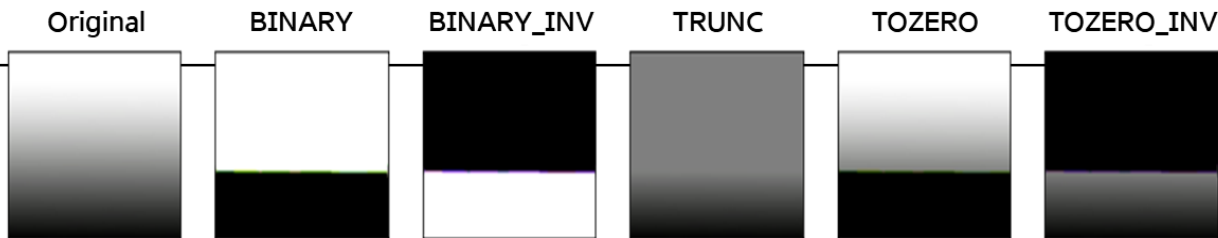
```
ret, thresh1 = cv2.threshold(org_img, 127, 255, cv2.THRESH_BINARY)
ret, thresh2 = cv2.threshold(org_img, 127, 255, cv2.THRESH_BINARY_INV)
ret, thresh3 = cv2.threshold(org_img, 127, 255, cv2.THRESH_TRUNC)
ret, thresh4 = cv2.threshold(org_img, 127, 255, cv2.THRESH_TOZERO)
ret, thresh5 = cv2.threshold(org_img, 127, 255, cv2.THRESH_TOZERO_INV)
```

임계치 적용방식을 다르게 적용

다음은 cv2 모듈의 threshold() 함수를 사용해 임계치로 회색조 이미지를 이진화하여 비교해 보자.

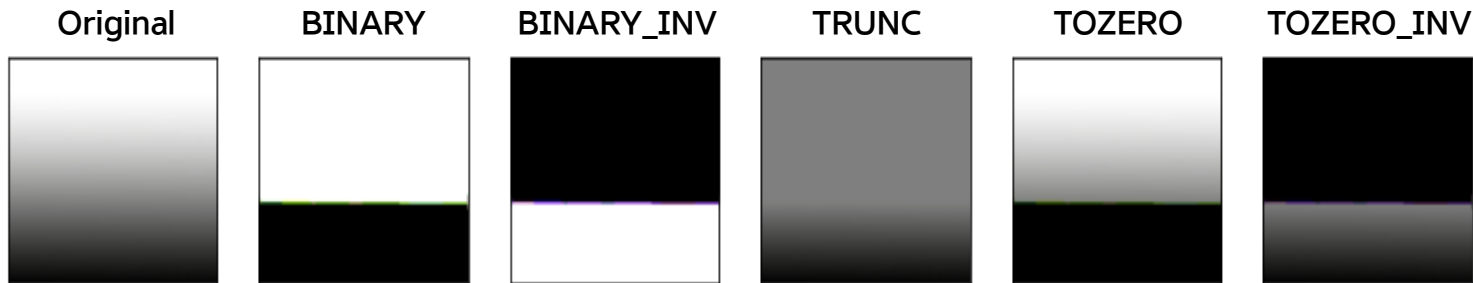
시각화하는 부분

```
titles=['Original','BINARY','BINARY_INV','TRUNC','TOZERO','TOZERO_INV']  
images=[org_img,thresh1,thresh2,thresh3,thresh4,thresh5]  
  
plt.figure(figsize=(15,15))  
for i in range(6):  
    plt.subplot(1,6,i+1), plt.imshow(images[i], 'gray'), plt.title(titles[i]), plt.xticks([]), plt.yticks([])  
  
plt.show()
```



임계치 적용방식이 다르게 적용된 결과

아래 그림은 임계값(=127)과 임계값을 넘었을 때 적용할 값(=255)의 적용 결과이다.



Thresh type	의미
THRESH_BINARY	조건을 만족하는 픽셀 값을 최댓값으로 만족하지 않는 픽셀 값을 0(검정색)으로 설정
THRESH_BINARY_INV	조건을 만족하는 픽셀 값을 0으로, 만족하지 않는 픽셀 값을 최대값으로 설정
THRESH_TRUNC	조건을 만족하는 픽셀 값을 최대값으로, 만족하지 않으면 원본 그대로 둠
THRESH_TOZERO	조건을 만족하는 픽셀 값을 원본 값으로, 만족하지 않는 픽셀 값을 0으로 설정
THRESH_TOZERO_INV	조건을 만족하는 픽셀 값을 0으로, 만족하지 않는 픽셀 값을 원본 값으로 설정

다음은 cv2 모듈의 `threshold()` 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(회색조 이미지 사용).



- 트랙바를 사용하여 임계치를 변경할 것이고, 트랙바에서 얻은 임계치로 이진화한 뒤 창에 그림을 그림
- 임계값을 높이면 기준을 만족하는 픽셀의 수가 적어지고, 높은 임계치를 주면 밝은 촛불 영역만 얻을 수 있음

다음은 cv2 모듈의 threshold() 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(회색조 이미지 사용).



임계치로 회색조 이미지를 이진화하여 창에 그림을 그림

임계값 타입 - THRESH_BINARY

조건을 만족하는 픽셀값을 최댓값으로 만족하지 않는 픽셀 값을 0(검정색)으로 설정함

다음은 cv2 모듈의 threshold() 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(회색조 이미지 사용).

구현 코드는 다음과 같음

```
import cv2, sys, os

global color_image, gray_image

# 트랙바가 변경되면 그 값을 임계치로 원본 컬러 이미지를 이진화하여 창에 그림
# - THRESH_BINARY: 조건을 만족하는 픽셀값을 최댓값으로 만족하지 않는 픽셀 값을 0(검정색)으로 설정
# - 회색조 이미지 사용
def on_change_threshold(x):
    _th_image = cv2.threshold(color_image, x, 255, cv2.THRESH_BINARY)
    cv2.imshow('Thresholding', th_image)

# 윈도우를 생성함
cv2.namedWindow('Thresholding')
cv2.createTrackbar('threshold', 'Thresholding', 0, 255, on_change_threshold)

# 촛불 이미지를 읽고, 회색조 이미지를 준비함
color_image = cv2.imread(os.getcwd()+'/images/candles.jpg', cv2.IMREAD_COLOR)
gray_image = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)

# 처음에는 원본 이미지를 그림(트랙바를 변경하면 임계치에 따라 이진화 결과 출력)
cv2.imshow('Thresholding', color_image)
cv2.waitKey() # 사용자 입력을 기다림
cv2.destroyAllWindows() # 모든 창을 없애고 프로그램 종료
```

트랙바 생성 및 임계값 적용

시각화하는 부분

다음은 cv2 모듈의 threshold() 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(회색조 이미지 사용).

트랙바 생성 및 임계값 적용

```
import cv2, sys, os

global color_image, gray_image

# 트랙바가 변경되면 그 값을 임계치로 원본 컬러 이미지를 이진화하여 창에 그림
# - THRESH_BINARY : 조건을 만족하는 픽셀값을 최댓값으로 만족하지 않는 픽셀 값을 0(검정색)으로 설정
# - 회색조 이미지 사용
def on_change_threshold(x):
    _, th_image = cv2.threshold(color_image, x, 255, cv2.THRESH_BINARY)
    cv2.imshow('Thresholding', th_image)

# 윈도우를 생성함
cv2.namedWindow('Thresholding')
cv2.createTrackbar('threshold', 'Thresholding', 0, 255, on_change_threshold)
```

다음은 cv2 모듈의 threshold() 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(회색조 이미지 사용).

시각화하는 부분

```
# 촛불 이미지를 읽고, 회색조 이미지를 준비함
color_image = cv2.imread(os.getcwd()+'/images/candles.jpg', cv2.IMREAD_COLOR)
gray_image = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)

# 처음에는 원본 이미지를 그림(트랙바를 변경하면 임계치에 따라 이진화 결과 출력)
cv2.imshow('Thresholding', color_image)
cv2.waitKey() # 사용자 입력을 기다림
cv2.destroyAllWindows() # 모든 창을 없애고 프로그램 종료
```

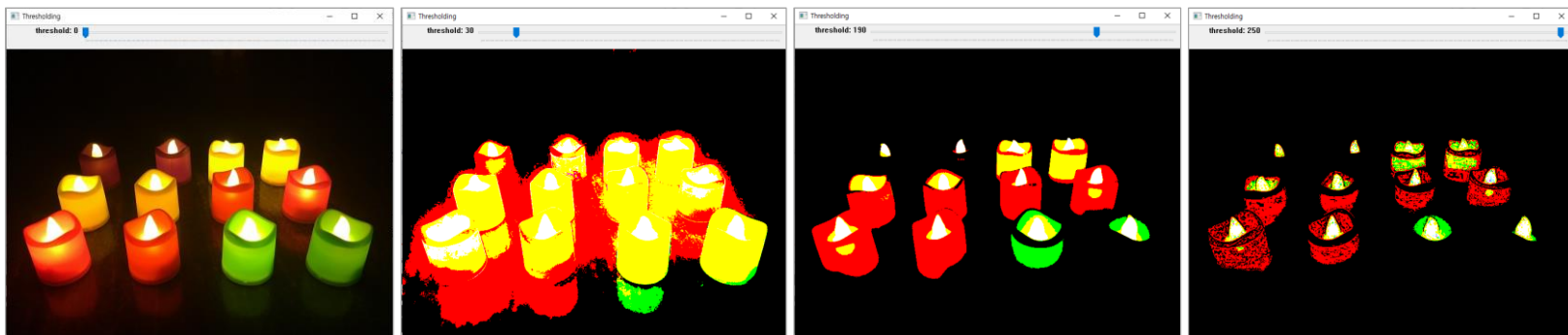


다음은 cv2 모듈의 threshold() 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(회색조 이미지 사용).



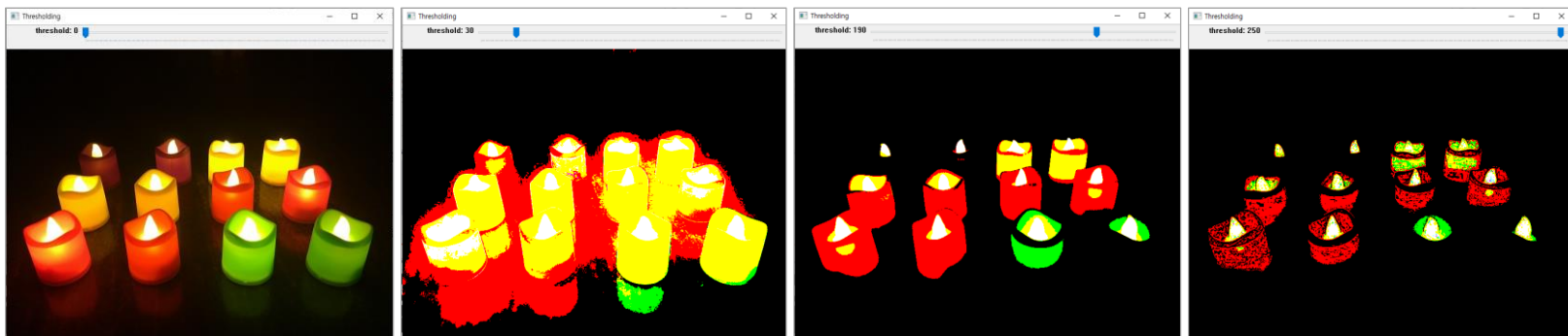
- 실행 결과에서 트랙바를 조절하여 임계값을 변경할 수 있음
- 임계값에 따라 통과하는 픽셀과 그렇지 못한 픽셀의 수가 바뀔 것임
- 임계값을 높이면 기준을 만족하는 픽셀의 수가 적어지고 적절히 높은 임계치를 주면 밝은 촛불 영역만 얻을 수 있는 것을 볼 수 있음

다음은 cv2 모듈의 `threshold()` 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(컬러 이미지 사용).



- 트랙바를 사용하여 임계치를 변경할 것이고, 트랙바에서 얻은 임계치로 이진화한 뒤 창에 그림을 그림
- 임계값을 높이면 기준을 만족하는 픽셀의 수가 적어지고, 높은 임계치를 주면 밝은 촛불 영역만 얻을 수 있음

다음은 cv2 모듈의 threshold() 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(컬러 이미지 사용).



임계치로 원본 이미지를 이진화하여 창에 그림을 그림

임계값 타입 - THRESH_BINARY

조건을 만족하는 픽셀값을 최댓값으로 만족하지 않는 픽셀 값을 0(검정색)으로 설정함

다음은 cv2 모듈의 threshold() 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(컬러 이미지 사용).

구현 코드는 다음과 같음

```
import cv2, sys, os

global color_image, gray_image

# 트랙바가 변경되면 그 값을 임계치로 원본 컬러 이미지를 이진화하여 창에 그림
# - THRESH_BINARY: 조건을 만족하는 픽셀값을 최댓값으로 만족하지 않는 픽셀 값을 0(검정색)으로 설정
# - 컬러 이미지 사용
def on_change_threshold(x):
    _th_image = cv2.threshold(color_image, x, 255, cv2.THRESH_BINARY)
    cv2.imshow('Thresholding', th_image)

# 윈도우를 생성함
cv2.namedWindow('Thresholding')
cv2.createTrackbar('threshold', 'Thresholding', 0, 255, on_change_threshold)

# 촛불 이미지를 읽고, 화색조 이미지를 준비함
color_image = cv2.imread(os.getcwd()+'/images/candles.jpg', cv2.IMREAD_COLOR)
gray_image = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)

# 처음에는 원본 이미지를 그림(트랙바를 변경하면 임계치에 따라 이진화 결과 출력)
cv2.imshow('Thresholding', color_image)
cv2.waitKey() # 사용자 입력을 기다림
cv2.destroyAllWindows() # 모든 창을 없애고 프로그램 종료
```

트랙바 생성 및 임계값 적용

시각화하는 부분

다음은 cv2 모듈의 threshold() 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(컬러 이미지 사용).

트랙바 생성 및 임계값 적용

```
import cv2, sys, os

global color_image, gray_image

# 트랙바가 변경되면 그 값을 임계치로 원본 컬러 이미지를 이진화하여 창에 그림
# - THRESH_BINARY : 조건을 만족하는 픽셀값을 최댓값으로 만족하지 않는 픽셀 값을 0(검정색)으로 설정
# - 컬러 이미지 사용
def on_change_threshold(x):
    _, th_image = cv2.threshold(color_image, x, 255, cv2.THRESH_BINARY)
    cv2.imshow('Thresholding', th_image)

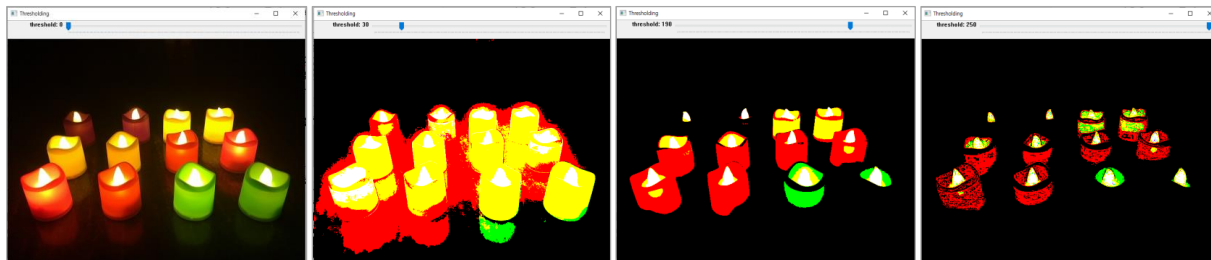
# 윈도우를 생성함
cv2.namedWindow('Thresholding')
cv2.createTrackbar('threshold', 'Thresholding', 0, 255, on_change_threshold)
```

다음은 cv2 모듈의 threshold() 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(컬러 이미지 사용).

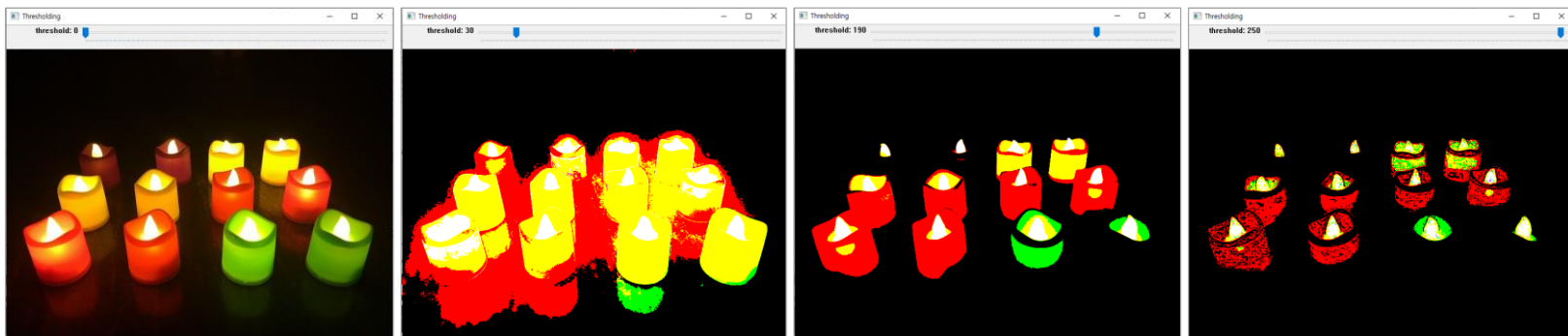
시각화하는 부분

```
# 촛불 이미지를 읽고, 회색조 이미지를 준비함
color_image = cv2.imread(os.getcwd()+'/images/candles.jpg', cv2.IMREAD_COLOR)
gray_image = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)

# 처음에는 원본 이미지를 그림(트랙바를 변경하면 임계치에 따라 이진화 결과 출력)
cv2.imshow('Thresholding', color_image)
cv2.waitKey() # 사용자 입력을 기다림
cv2.destroyAllWindows() # 모든 창을 없애고 프로그램 종료
```



다음은 cv2 모듈의 threshold() 함수를 이용해 여러 개의 초가 켜져 있는 이미지에서 촛불만 추출해 보자(컬러 이미지 사용).



- 실행 결과에서 트랙바를 조절하여 임계값을 변경할 수 있음
- 임계값에 따라 통과하는 픽셀과 그렇지 못한 픽셀의 수가 바뀔 것임
- 임계값을 높이면 기준을 만족하는 픽셀의 수가 적어지고 적절히 높은 임계치를 주면 밝은 촛불 영역만 얻을 수 있는 것을 볼 수 있음

이미지 윤곽선을 더 잘 뽑아 내기

앞에서 소개한(`cv2.threshold()` 함수) 임계치 적용 방식은
픽셀이 주위의 픽셀의 값과 어떤 관계를 가지는지 고려하지 않음

- ❖ 즉, 임계값을 이미지 전체에 적용하여 처리하기 때문에 하나의 이미지에 음영이 다르면 일부 영역이 모두 흰색 또는 검정색으로 보여지게 됨
- ❖ 그런데, 때때로 고정된 임계값이 아니라 주변 픽셀들이 가진 값을 기준으로 임계값을 정할 때도 있음

예) 이미지의 윤곽선을 찾는 것

이미지 윤곽선을 더 잘 뽑아 내기

앞에서 소개한(`cv2.threshold()` 함수) 임계치 적용 방식은 픽셀이 주위의 픽셀의 값과 어떤 관계를 가지는지 고려하지 않음

이미지의 윤곽선을 찾는 것

└─ 주위에 비해 튀는 값을 가진 픽셀

- ⌚ 주변 값에 따라 임계값을 정해야 함
- ⌚ 이런 임계값을 적응적(Adaptive) 임계값(Threshold)이라 함
- ⌚ 이런 문제에 사용할 수 있는 함수가 바로 CV2 모듈의 `adaptivethreshold()`

다음은 cv2 모듈의 `adaptiveThreshold()` 함수의 사용법이다.

```
cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)
```

- ➡ `src` : Grayscale image(입력 이미지)
- ➡ `maxValue` : 임계값(조건을 만족하는 픽셀에 적용할 최대값)
- ➡ `adaptiveMethod` : thresholding value를 결정하는 계산 방법(임계치를 결정하는 방법)
- ➡ `thresholdType` : threshold type(TRESH_BINARY와 TRESH_BINARY_INV 둘 중에서 하나가 사용, threshold() 함수에 사용된 것과 동일)
- ➡ `blockSize` : thresholding을 적용할 영역 사이즈(평균값을 구하는 범위를 결정)
- ➡ `C` : 평균이나 가중평균에서 차감할 값(임계치를 보정하는 데에 사용)

다음은 임계치를 결정하는 방법이다.

임계치를 결정하는 방법	의미
ADAPTIVE_THRESH_MEAN_C	픽셀 중심으로 $\text{blocksize} \times \text{blocksize}$ 구역 내의 픽셀 평균값 - C(마지막 인자)를 임계치로 함
ADAPTIVE_THRESH_GAUSSIAN_C	- 픽셀 중심으로 $\text{blocksize} \times \text{blocksize}$ 구역 내의 픽셀에 가우스 가중치를 적용한 평균값에서 C(마지막 인자)값을 뺀 값을 임계치로 함 - 가우스 가중치를 적용하므로 현재 검사하는 픽셀에서 가까운 픽셀이 더 중요하게 취급됨

다음은 cv2 모듈의 `adaptiveThreshold()` 함수를 이용해 이미지의 윤곽선을 추출해 보자.

- ➡ C(= -5, -1, 0, 1, 5)의 값을 변경하면서 그려보자.
- ➡ `blocksize` 구역 내(여기서는 9)의 픽셀에 **평균값(ADAPTIVE_THRESH_MEAN_C)**을 계산

```
color_image = cv2.imread(os.getcwd()+'/images/green_back.png', cv2.IMREAD_COLOR)
image_gray = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)

def img_adaptiveThreshold(x):
    return cv2.adaptiveThreshold(src=image_gray, maxValue=255,
                                adaptiveMethod=cv2.ADAPTIVE_THRESH_MEAN_C,
                                thresholdType=cv2.THRESH_BINARY, blockSize=9, C=x)

image_edge_m5 = img_adaptiveThreshold(-5)
image_edge_m1 = img_adaptiveThreshold(-1)
image_edge_ze = img_adaptiveThreshold(0)
image_edge_p1 = img_adaptiveThreshold(1)
image_edge_p5 = img_adaptiveThreshold(5)

titles = ['Original','edge, C=-5','edge, C=-1','edge, C=0','edge, C=1','edge, C=5']
images = [image_gray,image_edge_m5,image_edge_m1,image_edge_ze,image_edge_p1,image_edge_p5]

plt.figure(figsize=(15,15))
for i in range(6):
    plt.subplot(1,6,i+1), plt.imshow(images[i], 'gray'), plt.title(titles[i]), plt.xticks([]), plt.yticks([])
plt.show()
```

임계값 적용

시각화하는 부분

다음은 cv2 모듈의 `adaptiveThreshold()` 함수를 이용해 이미지의 윤곽선을 추출해 보자.

- ➡ C(= -5, -1, 0, 1, 5)의 값을 변경하면서 그려보자.
- ➡ blocksize 구역 내(여기서는 9)의 픽셀에 **평균값(ADAPTIVE_THRESH_MEAN_C)**을 계산

```
color_image = cv2.imread(os.getcwd()+'/images/green_back.png', cv2.IMREAD_COLOR)
image_gray = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)
```

```
def img_adaptiveThreshold(x) :
    return cv2.adaptiveThreshold(src=image_gray, maxValue=255,
                                adaptiveMethod=cv2.ADAPTIVE_THRESH_MEAN_C,
                                thresholdType=cv2.THRESH_BINARY, blockSize=9, C=x)
```

```
image_edge_m5 = img_adaptiveThreshold(-5)
image_edge_m1 = img_adaptiveThreshold(-1)
image_edge_ze = img_adaptiveThreshold(0)
image_edge_p1 = img_adaptiveThreshold(1)
image_edge_p5 = img_adaptiveThreshold(5)
```

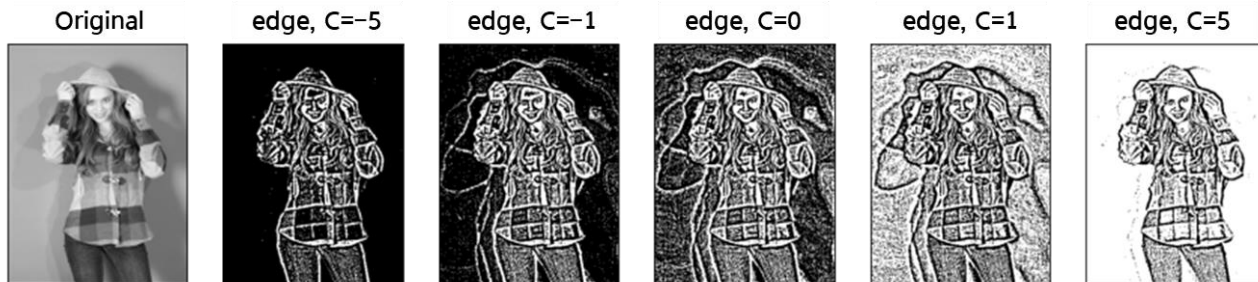
임계값 적용

다음은 cv2 모듈의 `adaptiveThreshold()` 함수를 이용해 이미지의 윤곽선을 추출해 보자.

시각화하는 부분

```
titles = ['Original', 'edge, C=-5', 'edge, C=-1', 'edge, C=0', 'edge, C=1', 'edge, C=5']
images = [image_gray, image_edge_m5, image_edge_m1, image_edge_ze, image_edge_p1, image_edge_p5]

plt.figure(figsize=(15,15))
for i in range(6):
    plt.subplot(1,6,i+1), plt.imshow(images[i], 'gray'), plt.title(titles[i]), plt.xticks([]), plt.yticks([])
plt.show()
```



아래 그림의 실행 결과에서 **blocksize** 구역 내(여기서는 9)의 픽셀에 **평균값**(ADAPTIVE_THRESH_MEAN_C)을 **계산**하고, **임계치**를 **보정**하는 값[-5, -1, 0, 1, 5]에 따라 **이미지**의 **윤곽선**이 **달라**지는 것을 볼 수 있다.

아래 그림에서 **임계치 보정값**이 5인 경우 이미지 **윤곽선**이 **가장 선명한** 것을 볼 수 있다.

Original



edge, C=-5



edge, C=-1



edge, C=0



edge, C=1



edge, C=5



픽셀에 평균값 계산

다음은 cv2 모듈의 `adaptiveThreshold()` 함수를 이용해 이미지의 윤곽선을 추출해 보자.

- ➡ C(= -5, -1, 0, 1, 5)의 값을 변경하면서 그려보자.
- ➡ `blocksize` 구역 내(여기서는 9)의 픽셀의 가우스 가중치(ADAPTIVE_THRESH_GAUSSIAN_C)를 계산

```
color_image = cv2.imread(os.getcwd()+'/images/green_back.png', cv2.IMREAD_COLOR)
image_gray = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)

def img_adaptiveThreshold(x):
    return cv2.adaptiveThreshold(src=image_gray, maxValue=255,
                                adaptiveMethod=cv2. ADAPTIVE_THRESH_GAUSSIAN_C,
                                thresholdType=cv2.THRESH_BINARY, blockSize=9, C=x)

image_edge_m5 = img_adaptiveThreshold(-5)
image_edge_m1 = img_adaptiveThreshold(-1)
image_edge_ze = img_adaptiveThreshold(0)
image_edge_p1 = img_adaptiveThreshold(1)
image_edge_p5 = img_adaptiveThreshold(5) plt.show()

titles = ['Original','edge, C=-5','edge, C=-1','edge, C=0','edge, C=1','edge, C=5']
images = [image_gray,image_edge_m5,image_edge_m1,image_edge_ze,image_edge_p1,image_edge_p5]

plt.figure(figsize=(15,15))
for i in range(6):
    plt.subplot(1,6,i+1), plt.imshow(images[i], 'gray'), plt.title(titles[i]), plt.xticks([]), plt.yticks([])
plt.show()
```

임계값 적용

시각화하는 부분

다음은 cv2 모듈의 `adaptiveThreshold()` 함수를 이용해 이미지의 윤곽선을 추출해 보자.

- ➡ C(= -5, -1, 0, 1, 5)의 값을 변경하면서 그려보자.
- ➡ `blocksize` 구역 내(여기서는 9)의 픽셀의 가우스 가중치(`ADAPTIVE_THRESH_GAUSSIAN_C`)를 계산

```
color_image = cv2.imread(os.getcwd()+'/images/green_back.png', cv2.IMREAD_COLOR)
image_gray = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)
```

```
def img_adaptiveThreshold(x) :
    return cv2.adaptiveThreshold(src=image_gray, maxValue=255,
                                adaptiveMethod=cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                thresholdType=cv2.THRESH_BINARY, blockSize=9, C=x)
```

```
image_edge_m5 = img_adaptiveThreshold(-5)
image_edge_m1 = img_adaptiveThreshold(-1)
image_edge_ze = img_adaptiveThreshold(0)
image_edge_p1 = img_adaptiveThreshold(1)
image_edge_p5 = img_adaptiveThreshold(5)
```

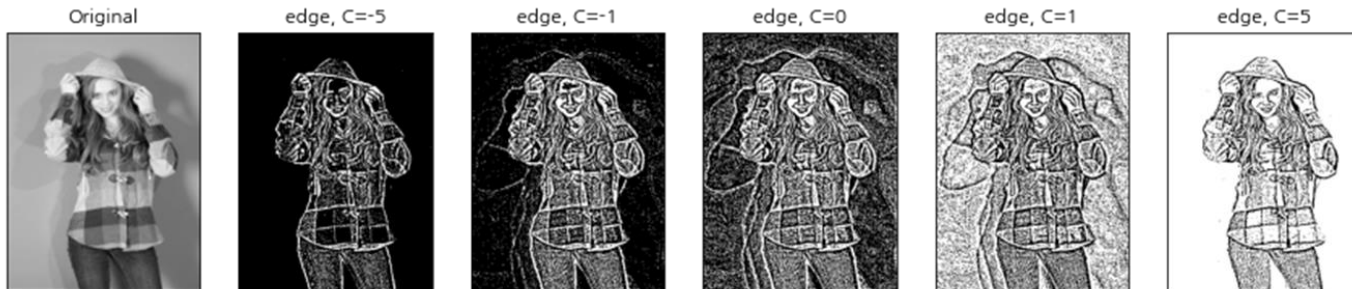
임계값 적용

다음은 cv2 모듈의 `adaptiveThreshold()` 함수를 이용해 이미지의 윤곽선을 추출해 보자.

시각화하는 부분

```
titles = ['Original', 'edge, C=-5', 'edge, C=-1', 'edge, C=0', 'edge, C=1', 'edge, C=5']
images = [image_gray, image_edge_m5, image_edge_m1, image_edge_ze, image_edge_p1, image_edge_p5]

plt.figure(figsize=(15,15))
for i in range(6):
    plt.subplot(1,6,i+1), plt.imshow(images[i], 'gray'), plt.title(titles[i]), plt.xticks([]), plt.yticks([])
plt.show()
```



아래 그림의 실행 결과에서 blocksize 구역 내(여기서는 9)의 픽셀에 가우스 가중치 (ADAPTIVE_THRESH_GAUSSIAN_C)을 계산하고, 임계치를 보정하는 값[-5, -1, 0, 1, 5]에 따라 이미지의 윤곽선이 달라지는 것을 볼 수 있다.

아래 그림에서 임계치 보정값이 5인 경우 이미지 윤곽선이 가장 선명한 것을 볼 수 있다.

Original



edge, C=-5



edge, C=-1



edge, C=0



edge, C=1



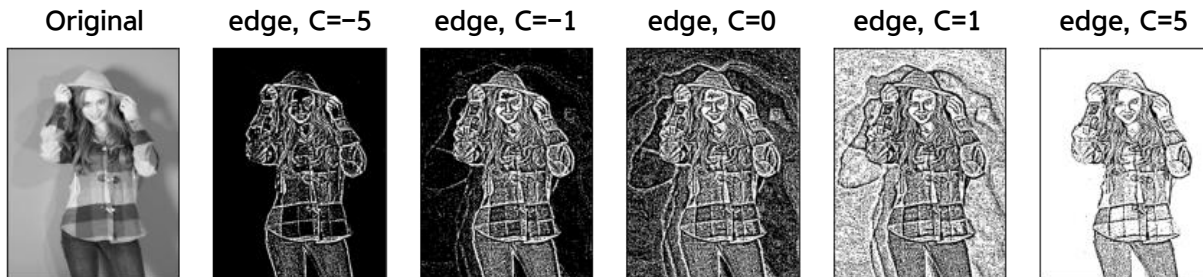
edge, C=5



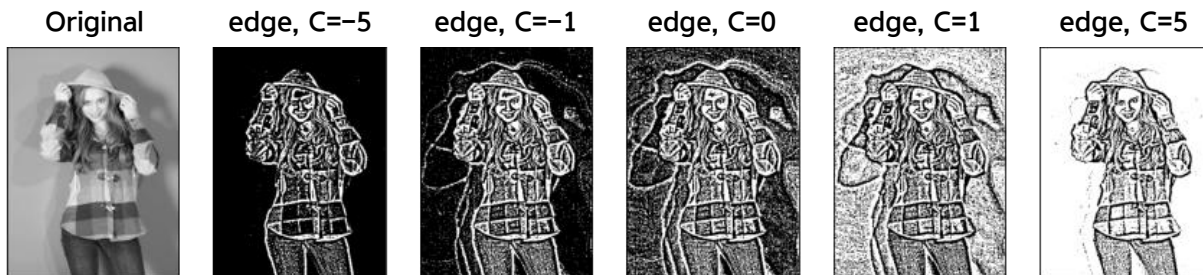
픽셀에 가우스 가중치 계산

아래의 두 실행결과를 비교해 보자.

픽셀에 가우스 가중치를 적용한 이미지의 윤곽선이 좀 더 선명한 것을 볼 수 있음



픽셀에 가우스 가중치 계산



픽셀에 평균값 계산

다음은 cv2 모듈의 `threshold()`, `adaptiveThreshold()` 함수를 사용해
이미지의 윤곽선을 추출해 보자.

- 여기서 `threshold()` 함수에는 임계값=127, 임계값을 넘었을 때 적용할 값은 255로 함
- `adaptiveThreshold()` 함수에는 픽셀에 평균값, 가우스 가중치를 적용함

```
color_image = cv2.imread(os.getcwd()+'/images/green_back.png', cv2.IMREAD_COLOR)
image_gray = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)

ret, th1 = cv2.threshold(image_gray, 127, 255, cv2.THRESH_BINARY)
th2 = cv2.adaptiveThreshold(image_gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 9, 5)
th3 = cv2.adaptiveThreshold(image_gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 9, 5)

titles = ['Original', 'threshold-Global', 'adaptiveThreshold-Mean-C5', 'adaptiveThreshold-Gaussian-C5']
images = [image_gray, th1, th2, th3]

plt.figure(figsize=(15,15))
for i in range(4):
    plt.subplot(1,4,i+1), plt.imshow(images[i], 'gray'), plt.title(titles[i]), plt.xticks([]), plt.yticks([])

plt.show()
```

아래 그림의 실행결과를 비교해 보자.

적응적(Adaptive) 임계값(Threshold)의 가우스 가중치를 적용한 이미지 윤곽선이
좀 더 선명한 것을 볼 수 있음

Original



Threshold-Global



adaptiveThreshold-Mean-C5



adaptiveThreshold-GAUSSIAN-C5



다음과 같이 녹색 배경에서 찍은 인물 사진과 배경으로 사용될 빙하 이미지가 있다.

인물 사진에서 녹색을 제거하고 다른 배경 사진과 합성하여
이 인물이 배경 사진이 찍힌 장소에 있는 것처럼 합성해 보자.



전경 이미지



배경 이미지



합성된 결과 이미지



다음과 같이 녹색 배경에서 찍은 인물 사진과
배경으로 사용될 이미지를 합성하는 코드이다.

```
img1 = cv2.imread(os.getcwd()+'/images/green_back.png', cv2.IMREAD_COLOR) # 전경 이미지 읽기
img2 = cv2.imread(os.getcwd()+'/images/iceberg.png', cv2.IMREAD_COLOR) # 배경 이미지 읽기
front_image = cv2.resize(img1, (300,400))
back_image = cv2.resize(img2, (300,400))

img_hsv = cv2.cvtColor(front_image, cv2.COLOR_BGR2HSV) # HSV 공간으로 옮김
l_bound = np.array([40, 100, 50]) # 녹색 색상의 하한
u_bound = np.array([80, 255, 255]) # 녹색 색상의 상한

grn_mask = cv2.inRange(img_hsv, l_bound, u_bound) # 녹색 픽셀 찾기
grn_mask_inv = cv2.bitwise_not(grn_mask) # 녹색이 아닌 픽셀 찾기

extracted = cv2.bitwise_and(front_image, front_image, mask=grn_mask) # 녹색 픽셀들만 추출하기
removed = cv2.bitwise_and(front_image, front_image, mask=grn_mask_inv) # 녹색 아닌 픽셀만 추출하기
background = cv2.bitwise_and(back_image, back_image, mask=grn_mask) # 녹색과 겹치는 배경 추출
merged = cv2.bitwise_or(removed, background) # 녹색 제거 전경 + 배경

cv2.imshow('front_image', front_image)
cv2.imshow('back_image', back_image)
cv2.imshow('grn_mask', grn_mask)
cv2.imshow('grn_mask_inv', grn_mask_inv)
cv2.imshow('extracted', extracted)
cv2.imshow('removed', removed)
cv2.imshow('background', background)
cv2.imshow('merged', merged)

cv2.waitKey() # 사용자 입력을 기다림
cv2.destroyAllWindows() # 모든 창을 없애고 프로그램 종료
```

이미지 합성

시각화하는 부분

다음과 같이 녹색 배경에서 찍은 인물 사진과
배경으로 사용될 이미지를 합성하는 코드이다.

이미지 합성

```
img1 = cv2.imread(os.getcwd()+'/images/green_back.png', cv2.IMREAD_COLOR) # 전경 이미지 읽기
img2 = cv2.imread(os.getcwd()+'/images/iceberg.png', cv2.IMREAD_COLOR) # 배경 이미지 읽기
front_image = cv2.resize(img1, (300,400))
back_image = cv2.resize(img2, (300,400))

img_hsv = cv2.cvtColor(front_image, cv2.COLOR_BGR2HSV) # HSV 공간으로 옮김
l_bound = np.array([40, 100, 50]) # 녹색 색상의 하한
u_bound = np.array([80, 255, 255]) # 녹색 색상의 상한

grn_mask = cv2.inRange(img_hsv, l_bound, u_bound) # 녹색 픽셀 찾기
grn_mask_inv = cv2.bitwise_not(grn_mask) # 녹색이 아닌 픽셀 찾기

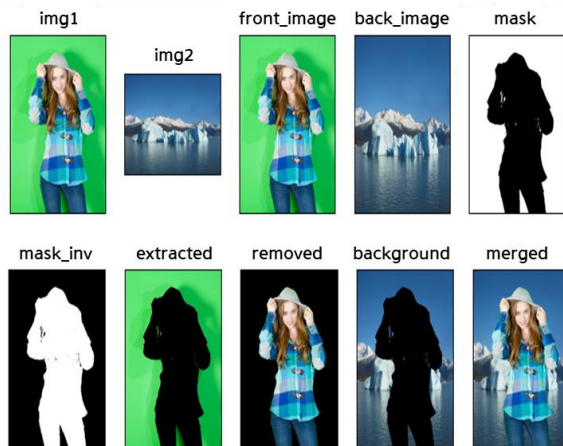
extracted = cv2.bitwise_and(front_image, front_image, mask=grn_mask) # 녹색 픽셀들만 추출하기
removed = cv2.bitwise_and(front_image, front_image, mask=grn_mask_inv) # 녹색 아닌 픽셀만 추출하기
background = cv2.bitwise_and(back_image, back_image, mask=grn_mask) # 녹색과 겹치는 배경 추출
merged = cv2.bitwise_or(removed, background) # 녹색 제거 전경 + 배경
```

다음과 같이 녹색 배경에서 찍은 인물 사진과
배경으로 사용될 이미지를 합성하는 코드이다.

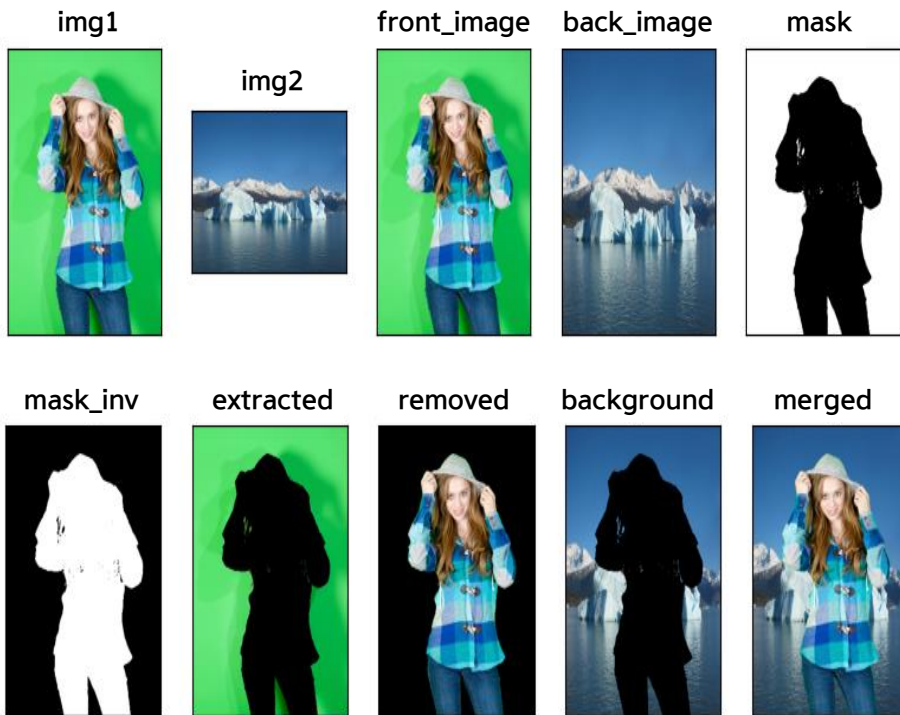
```
cv2.imshow('front_image', front_image)
cv2.imshow('back_image', back_image)
cv2.imshow('grn_mask', grn_mask)
cv2.imshow('grn_mask_inv', grn_mask_inv)
cv2.imshow('extracted', extracted)
cv2.imshow('removed', removed)
cv2.imshow('background', background)
cv2.imshow('merged', merged)

cv2.waitKey()          # 사용자 입력을 기다림
cv2.destroyAllWindows() # 모든 창을 없애고 프로그램 종료
```

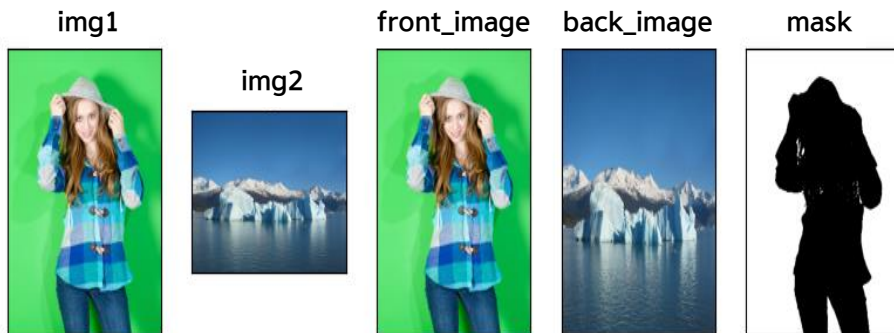
시각화하는 부분



아래 그림의 실행 결과를 비교 분석해 보자.



아래 그림의 실행 결과를 비교 분석해 보자.



- img1(front_image) : 전경 이미지
- img2(back_image) : 배경 이미지
- front_image : 전공 이미지
- back_image : 배경 이미지
- mask : 녹색 픽셀 찾기(녹색 색상의 하한: [40,100,50], 녹색 색상의 상한: [80,255,255])

아래 그림의 실행 결과를 비교 분석해 보자.



- mask_inv : 녹색이 아닌 픽셀 찾기
- extracted : 녹색 픽셀들만 추출
- removed : 녹색이 아닌 픽셀만 추출
- background : 녹색과 겹치는 배경 추출
- merged : 녹색을 제거한 전경과 배경 합성